R Programming Notes for Absolute Beginners

Part 1: Introduction, Variables and Data Types, with runnable code explained line by line

Written for people who have never programmed before. Take your time, type every example yourself, and be kind to yourself when something goes wrong. Everyone's code breaks at the start.

Contents

(If the contents list looks empty, right-click it in Word and choose Update Field.)

Part 1: Introduction to R

1.1 What is R, and why learn it?

Imagine you have a very fast, very obedient helper who never gets tired. The only catch is that the helper takes everything literally and only understands one language. That helper is your computer, and R is one of the languages you can use to talk to it.

R was built by statisticians, so it is brilliant at working with numbers, data, charts and maths. It is free, open source, and used by universities, hospitals, banks, governments and researchers all over the world.

You will hear two names, and they are not the same thing:

- **R** is the language and the engine that actually does the work.
- **RStudio** is the friendly dashboard that sits on top of R and makes it comfortable to use. Think of R as a car engine and RStudio as the steering wheel, dashboard and seats.

> **In plain words:** You install R first, then RStudio. Then you only ever open RStudio. If you would rather not install anything, a free browser version called Posit Cloud (posit.cloud) works too.

1.2 Getting set up (quick version)

- Download and install R from cloud.r-project.org (pick Windows, Mac or Linux).
- Download and install RStudio Desktop (free) from posit.co/download/rstudio-desktop.
- Open RStudio. You will see several panels. The most important two are the Console (where R answers) and the Source panel (where you write and save scripts).
- In RStudio choose File, then New File, then R Script. This opens an empty page where you will type the code in these notes.

1.3 How to read and use these notes

Every code example in these notes is shown in a table with three columns, so you always know exactly what is going on:

Column	What it is for
`Code (type this)`	The exact line to type into RStudio. Type it yourself instead of copying; your fingers learn too.
`What it does`	A plain-language explanation of that one line, as if a friend were sitting next to you.
`What you will see`	The result R should give you, so you can check you are on track.

A good routine for every example is: Read it, Predict what will happen, Run it, then Change something and run it again. Changing things and seeing what breaks is the fastest way to learn.

Run code in a script by placing your cursor on the line and pressing Ctrl + Enter (Windows/Linux) or Cmd + Enter (Mac). R runs that line and shows the answer in the Console.

> **Tip:** The snippets in each section build on each other. Run them from top to bottom. A ready-made R file with all the code from these notes comes with this document, so you can run everything at once.

1.4 Your very first lines of code

Code (type this)	What it does, in simple words	What you will see
`# This is a comment`	Any line starting with # is a note for humans. R ignores it completely. Use comments to explain your code to your future self.	*(nothing is shown)*
`print("Hello, R!")`	print() is a function: a little machine that shows something on screen. The words go inside quotation marks, because R must know it is text.	`[1] "Hello, R!"`
`2 + 3`	R works as a calculator. You can type maths straight in.	`[1] 5`
`10 / 4`	The / symbol means divide.	`[1] 2.5`
`2 * 3 + 4`	R follows the normal maths order: multiply first, then add. So this is 6 + 4.	`[1] 10`
`2 * (3 + 4)`	Brackets change the order. R works out the brackets first: 2 * 7.	`[1] 14`
`sqrt(16)`	sqrt() is a function that finds a square root. The number you give it goes inside the brackets.	`[1] 4`
`?sqrt`	A question mark in front of a function name opens its help page. Use this whenever you wonder what a function does.	`(help page opens)`

What is that [1] at the start of every answer?

It simply means "here comes item number 1 of the answer." R always answers with a list of values, even if there is just one. When the answer is long, R adds [13], [25] and so on to the start of each new line to help you count. You can safely ignore it for now.

Errors, warnings and messages: three different things

Type	What it means	What to do
`Error`	R could not do what you asked, and it stopped.	Read the message carefully. It usually names the problem, such as a typo or a missing bracket.
`Warning`	R did it, but something looks suspicious.	Check your result. Often it is harmless, but do not ignore it.
`Message`	Just information from R.	Read it and carry on.

> **In plain words:** An error is not you failing. It is R saying "I did not understand this, can you check it?" Most beginner errors come from typing mistakes: a missing quote, a missing bracket, or a capital letter in the wrong place.

Part 2: Variables

2.1 What is a variable?

A variable is a labelled box. You put something inside (a number, some text, whatever you like) and stick a name on the box. Later, instead of remembering the value, you just call the box by its name.

In R, you put a value into a box using the arrow <- . Read it as "gets" or "is assigned". So age <- 25 reads as "age gets 25".

Code (type this)	What it does, in simple words	What you will see
`age <- 25`	Makes a box called age and puts the number 25 inside. The arrow points from the value to the name.	*(nothing is shown)*
`age`	Typing just the name asks R to open the box and show what is inside.	`[1] 25`
`name <- "Amina"`	A box called name that holds text. Text always goes inside quotation marks.	*(nothing is shown)*
`name`	Look inside the name box.	`[1] "Amina"`
`is_student <- TRUE`	A box holding TRUE. TRUE and FALSE (in capitals, no quotes) are R's yes and no.	*(nothing is shown)*
`height = 1.65`	The equals sign also puts a value in a box. It works, but most R programmers prefer <- so we will stick with the arrow.	*(nothing is shown)*
`30 -> years`	The arrow can point the other way too. Rare, but good to recognise if you see it.	*(nothing is shown)*
`years`	Look inside years.	`[1] 30`

> **Tip:** RStudio has a shortcut for the arrow: hold Alt and press the minus key (Option and minus on a Mac). It types <- for you.

2.2 Showing what is inside a variable

There are several ways to see the contents of a variable. Each has its own personality.

Code (type this)	What it does, in simple words	What you will see
`print(age)`	The same as typing age, but written as a function. Useful inside bigger programs.	`[1] 25`
`cat("My name is", name, "and I am", age, "years old\n")`	cat() joins pieces together and prints them as a neat sentence, without the [1] and quote marks. The \n at the end means "start a new line".	`My name is Amina and I am 25 years old`
`paste("Hello", name)`	paste() glues pieces of text together with a space in between, and gives you the result as a new piece of text.	`[1] "Hello Amina"`
`paste("Hello", name, sep = "-")`	sep means separator. Here we glue with a dash instead of a space.	`[1] "Hello-Amina"`
`paste0("User", 1)`	paste0() glues with nothing in between.	`[1] "User1"`

Code (type this)	What it does, in simple words	What you will see
	Handy for making IDs such as User1, User2.	
`message("Just a friendly note")`	message() prints a note to you in the Console. It is meant for information, not for results.	`Just a friendly note`

2.3 Changing and copying variables

A box can be emptied and refilled at any time. That is why it is called a variable: its value can vary.

Code (type this)	What it does, in simple words	What you will see
`score <- 50`	Create the box score with 50 inside.	*(nothing is shown)*
`score <- score + 10`	Read the right side first: take what is in score (50), add 10, and put the answer (60) back into score. The old value is replaced.	*(nothing is shown)*
`score`	Check the new value.	`[1] 60`
`counter <- 0`	Start a counter at zero.	*(nothing is shown)*
`counter <- counter + 1`	Add one to the counter. Programmers do this all the time to count things.	*(nothing is shown)*
`counter <- counter + 1`	Add one again.	*(nothing is shown)*
`counter`	After two additions the counter is 2.	`[1] 2`
`a <- 5`	Box a holds 5.	*(nothing is shown)*
`b <- a`	Copy the contents of a into a new box b. Now both hold 5, but they are separate boxes.	*(nothing is shown)*
`a <- 100`	Change a only.	*(nothing is shown)*
`b`	b did not change, because it received its own copy.	`[1] 5`
`x <- y <- 7`	Fill two boxes with the same value in one go. R fills y first, then x.	*(nothing is shown)*
`c(x, y)`	c() combines values into a list. Both x and y hold 7.	`[1] 7 7`
`thing <- 5`	Put a number in a box.	*(nothing is shown)*
`class(thing)`	class() asks "what kind of thing is inside?"	`[1] "numeric"`
`thing <- "five"`	Replace the number with text. R does not mind, because the box can hold any type.	*(nothing is shown)*
`class(thing)`	The kind of thing inside has changed.	`[1] "character"`

A classic puzzle: how do you swap what is in two boxes? You need a third, empty box to hold one value while you move the other.

Code (type this)	What it does, in simple words	What you will see
`first <- 1`	Box first holds 1.	*(nothing is shown)*
`second <- 2`	Box second holds 2.	*(nothing is shown)*
`temp <- first`	Put a copy of first in a spare box so we do not lose it.	*(nothing is shown)*
`first <- second`	Now first holds 2.	*(nothing is shown)*
`second <- temp`	Put the saved 1 into second.	*(nothing is shown)*
`c(first, second)`	The two values have swapped places.	`[1] 2 1`

2.4 Using variables in calculations

The real power of variables: write the calculation once, then change the numbers whenever you like.

Code (type this)	What it does, in simple words	What you will see
`length_m <- 8`	The length of a room in metres.	*(nothing is shown)*
`width_m <- 5`	The width of the room.	*(nothing is shown)*
`area <- length_m * width_m`	Multiply the two boxes and store the answer in a new box called area.	*(nothing is shown)*
`area`	Look at the area.	`[1] 40`
`perimeter <- 2 * (length_m + width_m)`	Add length and width first (brackets), then double it.	*(nothing is shown)*
`perimeter`	The distance around the room.	`[1] 26`
`price <- 120; quantity <- 3`	A semicolon lets you put two commands on one line. Use it sparingly; separate lines are easier to read.	*(nothing is shown)*
`total <- price * quantity; total`	Calculate the total, then show it.	`[1] 360`

> **Tip:** Try changing length_m to 10 and run the area lines again. Everything updates. That is why variables beat typing numbers directly.

2.5 Naming rules and good style

You can name your boxes almost anything, but R has a few firm rules, and programmers have a few friendly habits.

Rule	Details
`Use letters, numbers, dots and underscores`	student_age, student.age and age2 are all fine.
`Never start with a number`	2age is not allowed, but age2 is.
`No spaces`	Use student_age instead of student age.
`Capital letters matter`	Age and age are two different boxes.
`Avoid reserved words`	TRUE, FALSE, if, else, for, function and NULL already have a meaning.
`Choose meaningful names`	total_price tells a story. x1 does not.

Rule	Details
`Pick one style`	Most R programmers use snake_case: all lowercase, words joined by underscores.

Code (type this)	What it does, in simple words	What you will see
`student_age <- 20`	A good name: lowercase with an underscore (snake_case).	*(nothing is shown)*
`studentAge <- 20`	Also valid. This style is called camelCase.	*(nothing is shown)*
`student.age <- 20`	Also valid. Dots are allowed in R names.	*(nothing is shown)*
`age2 <- 21`	Numbers are fine as long as they are not first.	*(nothing is shown)*
`Age <- 30`	A box called Age, with a capital A.	*(nothing is shown)*
`age <- 25`	A different box called age, with a small a.	*(nothing is shown)*
`Age`	R treats the two names as different boxes.	`[1] 30`
`age`	The small-a box still holds 25.	`[1] 25`
`2age <- 5`	Not allowed: names cannot begin with a number. R complains. (We leave this line out of the ready-made R file.)	`Error: unexpected symbol in "2age"`
`my age <- 5`	Not allowed: spaces split a name in two. R gets confused.	`Error: unexpected symbol in "my age"`
`if <- 5`	Not allowed: if is a reserved word that R uses for decisions.	`Error: unexpected assignment in "if <-"`

> **Watch out:** The exact wording of error messages can differ a little between R versions. What matters is learning to spot the cause.

2.6 Managing your variables

As you work, your workspace (R's memory) fills up with boxes. Here is how to look around, and tidy up.

Code (type this)	What it does, in simple words	What you will see
`ls()`	ls() means list. It shows the names of all the variables you have created so far.	`(a list of your variable names)`
`exists("age")`	Asks "is there a box called age?" Notice the name goes in quotes here.	`[1] TRUE`
`rm(score)`	rm() means remove. It throws away the box called score.	*(nothing is shown)*
`exists("score")`	Check that score is gone.	`[1] FALSE`
`assign("city", "Nairobi")`	assign() creates a box using a name stored as text. Useful when names are built by code.	*(nothing is shown)*
`city`	Look inside the new box.	`[1] "Nairobi"`

Code (type this)	What it does, in simple words	What you will see
`get("city")`	get() is the opposite: give it a name as text and it fetches the contents.	`[1] "Nairobi"`
`var_name <- "age"`	Store the name of a box inside another box. Boxes within boxes!	*(nothing is shown)*
`get(var_name)`	Fetches whatever box is named inside var_name, which is age.	`[1] 25`

> **Watch out:** rm(list = ls()) empties your whole workspace in one go. It is handy for starting fresh, but there is no undo. Only use it when you mean it.

2.7 R's ready-made variables (built-in constants)

Code (type this)	What it does, in simple words	What you will see
`pi`	R already knows pi. You never need to type 3.14159 yourself.	`[1] 3.141593`
`LETTERS[1:5]`	LETTERS is a built-in box holding A to Z. The [1:5] part means "take items 1 to 5".	`[1] "A" "B" "C" "D" "E"`
`letters[1:3]`	The lowercase version.	`[1] "a" "b" "c"`
`month.abb[1:3]`	Built-in abbreviated month names.	`[1] "Jan" "Feb" "Mar"`
`month.name[11]`	Full month names; item 11 is November.	`[1] "November"`
`radius <- 3`	Your own variable, to use with the built-in pi.	*(nothing is shown)*
`pi * radius^2`	The area of a circle: pi times radius squared. The ^ symbol means "to the power of".	`[1] 28.27433`

2.8 Exploring a variable: ask it questions

Once something is in a box, you can ask R all sorts of questions about it. These are the functions you will use again and again.

Code (type this)	What it does, in simple words	What you will see
`age <- 25`	Make sure age holds 25 again.	*(nothing is shown)*
`class(age)`	What kind of thing is it, in everyday terms?	`[1] "numeric"`
`typeof(age)`	How does R store it inside the computer? "double" just means a number with decimals allowed.	`[1] "double"`
`length(age)`	How many values are in the box? Just one.	`[1] 1`
`str(age)`	str() gives a short summary of the structure. "num" means numeric.	`num 25`
`is.numeric(age)`	A yes/no question: is it a number?	`[1] TRUE`
`is.character(age)`	Is it text? No.	`[1] FALSE`

Code (type this)	What it does, in simple words	What you will see
`name <- "Amina"`	Make sure name holds text.	*(nothing is shown)*
`nchar(name)`	nchar() counts the characters in text. Amina has five letters.	`[1] 5`
`ages <- c(25, 30, 22)`	A box can hold many values. c() combines numbers into one list called a vector.	*(nothing is shown)*
`length(ages)`	How many values? Three.	`[1] 3`
`ages[2]`	Square brackets pick out an item by position. The second one is 30.	`[1] 30`
`mean(ages)`	The average of the three ages.	`[1] 25.66667`
`max(ages)`	The biggest value.	`[1] 30`
`summary(ages)`	A quick overview: smallest, largest, middle and average.	`Min. 1st Qu. Median Mean 3rd Qu. Max. 22.00 23.50 25.00 25.67 27.50 30.00`

Your variable-exploring toolkit

Function	The question it answers
`class(x)`	What kind of thing is x?
`typeof(x)`	How does R store x inside the computer?
`length(x)`	How many values does x hold?
`str(x)`	Give me a short summary of x's structure.
`is.numeric(x)`, `is.character(x)`, `is.logical(x)`	Is x this type? (yes/no answer)
`exists("x")`	Is there a variable called x?
`ls()`	What variables do I have?
`rm(x)`	Remove x.
`summary(x)`	Give me the main numbers about x.
`head(x), tail(x)`	Show me the first (or last) few values.

2.9 Common beginner mistakes with variables

- **Forgetting quotes around text.** name <- Amina makes R search for a box called Amina. Write "Amina" instead.
- **Typing the name differently.** Age is not age. R is strict about capital letters.
- **Using = when you meant ==.** A single = puts a value in a box; a double == asks "are these equal?"
- **Forgetting that the old value is replaced.** score <- 10 wipes out whatever score held before.

- **Using variables you have not created yet.** You will see: Error: object 'x' not found. Check your spelling, or run the line that creates it first.

2.10 Practice time

- Create variables for your name, age, and favourite number. Use cat() to print one friendly sentence about yourself.
- A shop sells pens at 35 shillings each. Store the price and the quantity (say 12) in variables and calculate the total cost.
- Make a variable for a temperature in Celsius (say 30). Convert it to Fahrenheit using the rule F = C * 9/5 + 32.
- Use ls() to see your variables, then remove one with rm(), and check with exists().

Part 3: Data Types

3.1 What is a data type?

Every value in R has a type, which tells R what kind of thing it is and what you are allowed to do with it. You can add two numbers, but it makes no sense to add two names. You can shout a word in capital letters, but you cannot shout the number 7.

A good habit is to ask, about any piece of data: what type is this? R will tell you with class() and typeof().

The main data types at a glance

Type	Example	class()	typeof()	Used for
Numeric	3.14, 42, -7.5	numeric	double	Measurements, money, any number
Integer	5L, 100L	integer	integer	Whole numbers, counting
Character	"hello", "Nairobi"	character	character	Text, names, labels
Logical	TRUE, FALSE	logical	logical	Yes/no, conditions
Complex	3 + 4i	complex	complex	Advanced maths
Raw	as.raw(255)	raw	raw	Low-level bytes (rare)
Date	as.Date("2026-11-05")	Date	double	Calendar dates
Factor	factor(c("a","b"))	factor	integer	Categories

> **In plain words:** Most of your early R life will use four types: numeric, character, logical, and (a bit later) factor. Do not try to memorise the rest now. Just know they exist.

3.2 Numeric: numbers with decimals

Numeric is the default type for any number you type in R, whether it is 5 or 5.25. Underneath, R stores it as a "double", which just means a number that can have decimals.

Creating and checking numeric values

Code (type this)	What it does, in simple words	What you will see
`n <- 42.5`	Put a number with a decimal into a box called n.	*(nothing is shown)*
`class(n)`	Ask R what kind of thing n is.	`[1] "numeric"`
`typeof(n)`	How R stores it inside.	`[1] "double"`
`is.numeric(n)`	Yes/no: is n a number?	`[1] TRUE`
`class(7)`	Even a whole number like 7 is numeric by default.	`[1] "numeric"`

Arithmetic: the maths operations

Code (type this)	What it does, in simple words	What you will see
`n + 10`	Addition.	`[1] 52.5`
`n - 2.5`	Subtraction.	`[1] 40`
`n * 2`	Multiplication uses the star symbol.	`[1] 85`
`n / 5`	Division.	`[1] 8.5`
`n ^ 2`	Power: n multiplied by itself. (You can also write n ** 2.)	`[1] 1806.25`
`17 %% 5`	Modulus: the remainder after dividing. 17 divided by 5 is 3 with 2 left over.	`[1] 2`
`17 %/% 5`	Integer division: how many whole times 5 fits into 17.	`[1] 3`
`10 %% 2 == 0`	A neat trick to test for even numbers: if the remainder after dividing by 2 is 0, the number is even.	`[1] TRUE`

Rounding and other maths functions

Code (type this)	What it does, in simple words	What you will see
`x <- -7.68`	A negative number to experiment with.	*(nothing is shown)*
`abs(x)`	Absolute value: removes the minus sign.	`[1] 7.68`
`round(x)`	Rounds to the nearest whole number.	`[1] -8`
`round(x, 1)`	The second number says how many decimal places to keep.	`[1] -7.7`
`ceiling(x)`	Rounds UP towards the bigger number (the ceiling). For -7.68 that is -7.	`[1] -7`
`floor(x)`	Rounds DOWN to the smaller number (the floor).	`[1] -8`
`trunc(x)`	Chops off the decimal part without rounding.	`[1] -7`
`signif(123456, 2)`	Keeps 2 significant digits.	`[1] 120000`
`sqrt(81)`	Square root.	`[1] 9`
`exp(1)`	The number e raised to the power 1. exp() is the exponential function.	`[1] 2.718282`
`log(100)`	The natural logarithm (base e).	`[1] 4.60517`
`log10(100)`	Logarithm base 10: "10 to what power gives 100?"	`[1] 2`
`log2(8)`	Logarithm base 2.	`[1] 3`
`log(8, base = 2)`	The same idea with a base you choose.	`[1] 3`
`factorial(5)`	5 x 4 x 3 x 2 x 1.	`[1] 120`
`choose(5, 2)`	How many ways to choose 2 things from 5.	`[1] 10`

Code (type this)	What it does, in simple words	What you will see
`sin(pi / 2)`	Trigonometry works in radians. The sine of 90 degrees is 1.	`[1] 1`
`max(3, 9, 2)`	The biggest of the values.	`[1] 9`
`min(3, 9, 2)`	The smallest.	`[1] 2`
`sum(1:10)`	Adds 1+2+...+10. The 1:10 part means "all whole numbers from 1 to 10".	`[1] 55`
`mean(c(2, 4, 6))`	The average.	`[1] 4`

Watch out: R rounds exactly-half numbers to the nearest EVEN number. So round(2.5) gives 2, round(3.5) gives 4, and round(0.5) gives 0. This surprises almost everyone the first time.

Showing numbers nicely

Code (type this)	What it does, in simple words	What you will see
`1e6`	Scientific notation. 1e6 means 1 times 10 to the power 6, which is a million. R prints big round numbers this way.	`[1] 1e+06`
`format(1e6, scientific = FALSE)`	format() turns a number into nicely written text. Here we switch off scientific notation.	`[1] "1000000"`
`format(1234567.891, big.mark = ",")`	Adds thousands separators.	`[1] "1,234,568"`
`round(3.14159, 2)`	Keep two decimal places.	`[1] 3.14`
`sprintf("%.2f", 3.14159)`	sprintf() builds text from a template. %.2f means "a number with 2 decimal places". Great when the output must look exactly right.	`[1] "3.14"`
`sprintf("%05d", 42L)`	%05d means "a whole number padded with zeros to 5 digits".	`[1] "00042"`

Special numbers: Inf, NaN and the precision puzzle

Code (type this)	What it does, in simple words	What you will see
`1 / 0`	Dividing by zero does not crash R. It gives infinity.	`[1] Inf`
`-1 / 0`	Negative infinity.	`[1] -Inf`
`0 / 0`	NaN means "Not a Number": a calculation with no sensible answer.	`[1] NaN`
`sqrt(-1)`	You cannot take the square root of a negative in real numbers. R gives NaN and a warning (Warning message: NaNs produced).	`[1] NaN`
`is.finite(5)`	Is it an ordinary, finite number?	`[1] TRUE`
`is.infinite(1 / 0)`	Is it infinity?	`[1] TRUE`
`is.nan(0 / 0)`	Is it NaN?	`[1] TRUE`

Code (type this)	What it does, in simple words	What you will see
`0.1 + 0.2 == 0.3`	Surprise! Computers store decimals in binary, so tiny errors sneak in. The two sides are not exactly equal.	`[1] FALSE`
`isTRUE(all.equal(0.1 + 0.2, 0.3))`	all.equal() checks "equal, give or take a tiny difference". This is the right way to compare decimals.	`[1] TRUE`
`sqrt(2)^2 == 2`	Another example of the same tiny-error effect.	`[1] FALSE`
`.Machine$double.eps`	The smallest difference R can reliably notice around 1.	`[1] 2.220446e-16`

> **In plain words:** Think of decimals on a computer like measuring with a ruler that only has millimetre marks. You can get very close to any length, but not always exactly right. So we compare decimals with "close enough", not "identical".

3.3 Integer: whole numbers

An integer is a whole number with no decimal part. To tell R you want an integer, put a capital L after the number.

Code (type this)	What it does, in simple words	What you will see
`i <- 5L`	The L means "make this a true integer".	*(nothing is shown)*
`class(i)`	Confirm the type.	`[1] "integer"`
`is.integer(5)`	Plain 5 is numeric, not integer.	`[1] FALSE`
`is.integer(5L)`	But 5L is an integer.	`[1] TRUE`
`as.integer(7.9)`	Converts to integer by chopping off the decimals (no rounding).	`[1] 7`
`as.integer("12")`	Converts text that looks like a number into an integer.	`[1] 12`
`5L + 3L`	Integer plus integer stays an integer.	`[1] 8`
`class(5L + 3L)`	Check it.	`[1] "integer"`
`5L / 2L`	Division can give decimals, so the answer becomes numeric.	`[1] 2.5`
`class(5L / 2L)`	Confirm the change.	`[1] "numeric"`
`5L %/% 2L`	Integer division keeps it an integer.	`[1] 2`
`1:5`	The colon creates a run of whole numbers.	`[1] 1 2 3 4 5`
`class(1:5)`	Ranges made with a colon are integers.	`[1] "integer"`
`.Machine$integer.max`	The biggest integer R can hold.	`[1] 2147483647`
`.Machine$integer.max + 1L`	Go beyond the limit and you get NA (a missing value) with a warning about integer overflow.	`[1] NA`

> **Tip:** For everyday work, plain numeric is fine. Integers matter mostly for counting, positions, and saving memory in big datasets.

3.4 Character: text

Text in R is called character data (also called a string). Anything inside quotation marks is text, even if it looks like a number. You can use double quotes or single quotes, as long as you close with the same kind.

Creating and checking text

Code (type this)	What it does, in simple words	What you will see
`greeting <- "Hello, World"`	Store a piece of text.	*(nothing is shown)*
`class(greeting)`	Ask for its type.	`[1] "character"`
`is.character(greeting)`	Yes/no check.	`[1] TRUE`
`'Single quotes work too'`	Single quotes are fine. R shows the result with double quotes.	`[1] "Single quotes work too"`
`class("123")`	This looks like a number, but the quotes make it text.	`[1] "character"`
`nchar(greeting)`	Counts characters, including spaces and punctuation.	`[1] 12`
`nchar("")`	An empty piece of text has zero characters.	`[1] 0`

Everything you can do with text

Code (type this)	What it does, in simple words	What you will see
`toupper(greeting)`	Turns every letter into a capital.	`[1] "HELLO, WORLD"`
`tolower(greeting)`	Turns every letter into lowercase.	`[1] "hello, world"`
`substr(greeting, 1, 5)`	Takes a slice: from the 1st character to the 5th.	`[1] "Hello"`
`paste("Good", "morning")`	Joins text with a space between.	`[1] "Good morning"`
`paste("a", "b", sep = "-")`	Joins with a separator of your choice.	`[1] "a-b"`
`paste0("ID", 1:3)`	Joins with no gap, and repeats for each number. Makes three labels at once.	`[1] "ID1" "ID2" "ID3"`
`paste(c("a", "b", "c"), collapse = "")`	collapse squashes a whole list of text into one single piece.	`[1] "abc"`
`strsplit("cat,dog,pig", ",")`	Splits text wherever it finds a comma. The answer comes back as a list.	`[[1]] [1] "cat" "dog" "pig"`
`unlist(strsplit("cat,dog,pig", ","))`	unlist() removes the list wrapper to leave a plain set of words.	`[1] "cat" "dog" "pig"`
`gsub("o", "0", "foo boo")`	Replaces every "o" with "0". The g stands for global (all of them).	`[1] "f00 b00"`

Code (type this)	What it does, in simple words	What you will see
`sub("o", "0", "foo boo")`	Replaces only the first match.	`[1] "f0o boo"`
`grepl("World", greeting)`	Does the text contain "World"? Answers TRUE or FALSE.	`[1] TRUE`
`startsWith(greeting, "Hello")`	Does it begin with "Hello"?	`[1] TRUE`
`endsWith(greeting, "x")`	Does it end with "x"?	`[1] FALSE`
`trimws("  hi  ")`	Trims away spaces at the start and end.	`[1] "hi"`
`toString(1:3)`	Turns a list of values into one piece of text, separated by commas.	`[1] "1, 2, 3"`
`rev(c("a", "b", "c"))`	Reverses the order of a list.	`[1] "c" "b" "a"`
`sort(c("pear", "apple", "mango"))`	Sorts text alphabetically.	`[1] "apple" "mango" "pear"`
`"apple" < "banana"`	Text can be compared alphabetically.	`[1] TRUE`
`"apple" == "Apple"`	Capitals matter, so these are not equal.	`[1] FALSE`

Building sentences with sprintf and special characters

Code (type this)	What it does, in simple words	What you will see
`sprintf("My name is %s and I am %d", "Amina", 25L)`	sprintf() fills a template. %s is a slot for text and %d is a slot for a whole number, filled in order.	`[1] "My name is Amina and I am 25"`
`sprintf("Price: %.2f", 19.5)`	%.2f is a slot for a decimal with 2 places.	`[1] "Price: 19.50"`
`cat("Line1\nLine2\n")`	\n means "new line". cat() obeys it, so the words appear on two lines.	`Line1` `Line2`
`cat("Name:\tAmina\n")`	\t means "tab": a wide space, handy for tidy columns.	`Name:  Amina`
`cat("She said \"Hi\"\n")`	A backslash before a quote tells R "this quote is part of the text".	`She said "Hi"`
`print("She said \"Hi\"")`	print() shows the backslash, because it displays R's view of the text.	`[1] "She said \"Hi\""`

A small challenge: reversing a word

Let us reverse the text "R is fun". We will do it in small steps so you can see each stage.

Code (type this)	What it does, in simple words	What you will see
`word <- "R is fun"`	The text to reverse.	*(nothing is shown)*
`letters_list <- strsplit(word, "")[[1]]`	Split into single characters (splitting on nothing). The [[1]] takes the list out of its wrapper.	*(nothing is shown)*
`rev(letters_list)`	Reverse the order of the characters.	`[1] "n" "u" "f" " " "s" "i" " " "R"`
`paste(rev(letters_list), collapse = "")`	Glue the reversed characters back together into one piece of text.	`[1] "nuf si R"`

3.5 Logical: TRUE or FALSE

A logical value answers a yes/no question. It is either TRUE or FALSE, written in capitals with no quotes. Logical values are the secret ingredient behind decisions in your code: "if this is TRUE, do that."

Code (type this)	What it does, in simple words	What you will see
`is_adult <- TRUE`	A box holding a logical value.	*(nothing is shown)*
`class(is_adult)`	Check the type.	`[1] "logical"`
`5 > 3`	Is 5 greater than 3? Comparisons always give TRUE or FALSE.	`[1] TRUE`
`5 < 3`	Is 5 less than 3?	`[1] FALSE`
`5 >= 5`	Greater than or equal to.	`[1] TRUE`
`5 == 3`	Double equals asks "are they equal?" (a single = would put a value in a box).	`[1] FALSE`
`5 != 3`	Not equal to. The ! means "not".	`[1] TRUE`

Combining yes/no answers

Code (type this)	What it does, in simple words	What you will see
`TRUE & FALSE`	AND: true only if BOTH sides are true.	`[1] FALSE`
`TRUE | FALSE`	OR: true if AT LEAST ONE side is true. (The | symbol is the vertical bar.)	`[1] TRUE`
`!TRUE`	NOT: flips the answer.	`[1] FALSE`
`xor(TRUE, FALSE)`	Exclusive OR: true if exactly one side is true.	`[1] TRUE`
`age <- 25`	A number to test.	*(nothing is shown)*
`age >= 18 & age <= 65`	Is the age between 18 and 65? Both checks must pass.	`[1] TRUE`
`age < 18 | age > 65`	Is the age outside the working range? Either check is enough.	`[1] FALSE`
`isTRUE(age > 18)`	A strict check: is the result exactly TRUE (and not something odd like NA)?	`[1] TRUE`

Logical values in lists, and logical as numbers

Behind the scenes, TRUE counts as 1 and FALSE counts as 0. That means you can add and average logical values, which is a lovely shortcut for counting things.

Code (type this)	What it does, in simple words	What you will see
`results <- c(TRUE, FALSE, TRUE, TRUE)`	A list of four yes/no answers.	*(nothing is shown)*
`sum(results)`	Counts how many are TRUE.	`[1] 3`

Code (type this)	What it does, in simple words	What you will see
`mean(results)`	The share that are TRUE: 3 out of 4.	`[1] 0.75`
`any(results)`	Is at least one of them TRUE?	`[1] TRUE`
`all(results)`	Are all of them TRUE?	`[1] FALSE`
`which(results)`	Positions of the TRUE values.	`[1] 1 3 4`
`!results`	Flips every answer in the list.	`[1] FALSE TRUE FALSE FALSE`
`scores <- c(40, 75, 62, 90)`	Some exam scores.	*(nothing is shown)*
`scores > 60`	Compares every score to 60 at once.	`[1] FALSE TRUE TRUE TRUE`
`scores[scores > 60]`	Use the TRUE/FALSE answers as a filter: keep only scores above 60.	`[1] 75 62 90`
`sum(scores > 60)`	How many scores are above 60?	`[1] 3`
`ifelse(scores >= 50, "Pass", "Fail")`	For each score, give "Pass" if the test is TRUE and "Fail" if not.	`[1] "Fail" "Pass" "Pass" "Pass"`
`as.numeric(TRUE)`	TRUE becomes 1.	`[1] 1`
`TRUE + TRUE`	Adding logical values works, because they act as 1s.	`[1] 2`

> **Watch out:** Always write TRUE and FALSE in full. R also accepts T and F, but those are ordinary variable names that someone could accidentally change, which causes confusing bugs.

3.6 Complex numbers (for the curious)

A complex number has a real part and an imaginary part, written with a lowercase i, such as 3 + 4i. You will rarely need them in beginner work, but R handles them without fuss, which is great for engineering and advanced maths.

Code (type this)	What it does, in simple words	What you will see
`z <- 3 + 4i`	Create a complex number: real part 3, imaginary part 4.	*(nothing is shown)*
`class(z)`	Check the type.	`[1] "complex"`
`Re(z)`	The real part.	`[1] 3`
`Im(z)`	The imaginary part.	`[1] 4`
`Mod(z)`	The size (modulus) of the number: sqrt(3^2 + 4^2).	`[1] 5`
`Conj(z)`	The conjugate: same real part, opposite imaginary part.	`[1] 3-4i`
`z + (1 + 2i)`	Adding complex numbers adds the real and imaginary parts separately.	`[1] 4+6i`
`z * 2`	Multiplying by a normal number.	`[1] 6+8i`
`sqrt(as.complex(-1))`	The square root of -1 is the famous imaginary number i. We must first tell R to treat -1 as complex.	`[1] 0+1i`

3.7 Raw: the computer's own alphabet (very brief)

Raw data is how a computer stores information as bytes, shown in a base-16 code called hexadecimal. As a beginner you can skip this, but here is a quick peek.

Code (type this)	What it does, in simple words	What you will see
`charToRaw("A")`	Shows the byte code behind the letter A (41 in hexadecimal).	`[1] 41`
`as.raw(255)`	The biggest single byte, shown as ff.	`[1] ff`
`rawToChar(as.raw(72))`	Turns byte code 72 back into a letter.	`[1] "H"`
`class(as.raw(1))`	The type name.	`[1] "raw"`

3.8 Dates and times

Dates look like text ("2026-11-05"), but R can treat them as real calendar dates. That lets you subtract two dates to count days, add weeks, find the day of the week, and more. The trick is to convert text into a Date first.

Code (type this)	What it does, in simple words	What you will see
`today <- Sys.Date()`	Sys.Date() asks the computer for today's date.	*(nothing is shown)*
`class(today)`	It has its own type: Date.	`[1] "Date"`
`d <- as.Date("2026-11-05")`	as.Date() turns text into a real date. The standard order is year-month-day.	*(nothing is shown)*
`d`	Display the date.	`[1] "2026-11-05"`
`format(d, "%d/%m/%Y")`	format() changes how the date looks. %d is day, %m is month, %Y is the 4-digit year.	`[1] "05/11/2026"`
`format(d, "%A")`	%A gives the full weekday name.	`[1] "Thursday"`
`weekdays(d)`	A shortcut for the weekday name.	`[1] "Thursday"`
`months(d)`	The month name.	`[1] "November"`
`format(d, "%Y")`	Just the year (as text).	`[1] "2026"`
`d + 30`	Adding a number adds that many days.	`[1] "2026-12-05"`
`as.Date("2026-12-25") - d`	Subtract two dates to find the gap between them.	`Time difference of 50 days`
`as.numeric(as.Date("2026-12-25") - d)`	Turn the difference into a plain number.	`[1] 50`
`d > as.Date("2026-01-01")`	Dates can be compared like numbers.	`[1] TRUE`
`seq(d, by = "month", length.out = 3)`	Make a sequence of 3 dates, one month apart.	`[1] "2026-11-05" "2026-12-05" "2027-01-05"`
`as.Date("05/11/2026", format = "%d/%m/%Y")`	If your date is written day/month/year, tell R the pattern with format.	`[1] "2026-11-05"`

Code (type this)	What it does, in simple words	What you will see
`typeof(d)`	Under the hood a Date is a number: days counted since 1 January 1970.	`[1] "double"`
`class(Sys.time())`	Sys.time() gives the exact date and time. Its type has two names.	`[1] "POSIXct" "POSIXt"`

Code	Meaning	Example
%d	Day of the month (two digits)	05
%m	Month number	11
%Y	Year with four digits	2026
%y	Year with two digits	26
%B	Full month name	November
%b	Short month name	Nov
%A	Full weekday name	Thursday
%a	Short weekday name	Thu

3.9 Factors: categories

A factor is R's way of storing categories: things that belong to a small set of groups, like "small / medium / large", "male / female", or "low / medium / high". R remembers the full list of allowed groups, called levels.

Code (type this)	What it does, in simple words	What you will see
`sizes <- factor(c("small", "large", "medium", "small"))`	Creates a factor from four pieces of text. R spots the three different groups.	*(nothing is shown)*
`sizes`	Shows the values and the levels (the groups, in alphabetical order by default).	`[1] small  large medium small` `Levels: large medium small`
`levels(sizes)`	Lists just the groups.	`[1] "large" "medium" "small"`
`nlevels(sizes)`	How many groups?	`[1] 3`

`table(sizes)`	Counts how many items fall in each group.	`sizes`
		` large medium small`
		`     1      1      2`

`as.integer(sizes)`	Behind the scenes, each group is stored as a number: large = 1, medium = 2, small = 3.	`[1] 3 1 2 3`
`grade <- factor(c("low", "high", "medium"), levels = c("low", "medium", "high"), ordered = TRUE)`	An ordered factor: we give the levels in the order we want, and ordered = TRUE says that order matters.	*(nothing is shown)*
`grade`	Notice the less-than signs showing the order.	`[1] low    high   medium` `Levels: low <`

Code (type this)	What it does, in simple words	What you will see
		`medium < high`
`grade[2] > grade[1]`	With an ordered factor you can compare. Is "high" above "low"?	`[1] TRUE`
`class(grade)`	An ordered factor has two class names.	`[1] "ordered" "factor"`

Watch out: A classic trap: if numbers got stored as a factor, as.numeric() gives you the hidden group codes, not the original numbers. Convert via text first, as shown below.

Code (type this)	What it does, in simple words	What you will see
`f <- factor(c("10", "20", "10"))`	A factor that looks like numbers.	*(nothing is shown)*
`as.numeric(f)`	Wrong! This returns the hidden codes (1 for "10", 2 for "20").	`[1] 1 2 1`
`as.numeric(as.character(f))`	Right: first turn it back into text, then into numbers.	`[1] 10 20 10`

3.10 Special values: NA, NULL, NaN and Inf

Real data is messy. Sometimes a value is missing, and R has special words for different kinds of "nothing":

Value	Meaning	Example
`NA`	Not Available: a value is missing or unknown	A survey question nobody answered
`NULL`	Nothing at all; an empty object	An empty result
`NaN`	Not a Number: a maths result that makes no sense	0 / 0
`Inf`	Infinity	1 / 0

Code (type this)	What it does, in simple words	What you will see
`x <- c(5, NA, 8)`	A list of three numbers where the middle one is missing.	*(nothing is shown)*
`is.na(x)`	Asks of each value: "is it missing?"	`[1] FALSE TRUE FALSE`
`sum(x)`	With a missing value, R refuses to guess and answers NA.	`[1] NA`
`sum(x, na.rm = TRUE)`	na.rm = TRUE means "remove the NAs before calculating".	`[1] 13`
`mean(x, na.rm = TRUE)`	The average of the values that exist.	`[1] 6.5`
`sum(is.na(x))`	Counts how many values are missing (TRUE counts as 1).	`[1] 1`
`which(is.na(x))`	Which positions are missing?	`[1] 2`
`x[!is.na(x)]`	Keep only the values that are NOT missing.	`[1] 5 8`

Code (type this)	What it does, in simple words	What you will see
`ifelse(is.na(x), 0, x)`	Replace each missing value with 0 and keep the others.	`[1] 5 0 8`
`NA + 1`	Anything combined with an unknown stays unknown.	`[1] NA`
`NA == NA`	Is unknown equal to unknown? Nobody can tell!	`[1] NA`
`NA & FALSE`	Whatever NA is, FALSE and anything is FALSE, so R knows the answer.	`[1] FALSE`
`NA | TRUE`	Likewise, TRUE or anything is TRUE.	`[1] TRUE`
`class(NA)`	A plain NA counts as logical.	`[1] "logical"`
`class(NA_character_)`	A missing piece of text has its own version of NA.	`[1] "character"`
`length(NULL)`	NULL has nothing inside, so its length is zero.	`[1] 0`
`is.null(NULL)`	Is it NULL?	`[1] TRUE`
`is.na(NaN)`	NaN counts as missing for is.na().	`[1] TRUE`
`is.nan(NA)`	But NA is not a NaN.	`[1] FALSE`
`Inf - Inf`	Infinity minus infinity is not a sensible number.	`[1] NaN`

> **Tip:** Never test for missing values with x == NA. The answer is always NA. Use is.na(x) instead.

3.11 Checking data types: your detective toolkit

Here is the full set of tools for finding out what a piece of data is. Use whichever answers your question.

Function	What it tells you
`class(x)`	The type as a user sees it (the most useful one)
`typeof(x)`	How R stores it internally
`mode(x)`	A broad type: numeric, character, logical, ...
`storage.mode(x)`	Like typeof(), for basic storage
`str(x)`	A compact summary of the type and contents
`is.numeric(x)`, `is.integer(x)`, `is.character(x)`, `is.logical(x)`, `is.factor(x)`	Yes/no questions about type
`inherits(x, "numeric")`	Does x belong to this class?

Code (type this)	What it does, in simple words	What you will see
`v <- 3.5`	A number to inspect.	*(nothing is shown)*
`class(v)`	Everyday type.	`[1] "numeric"`

Code (type this)	What it does, in simple words	What you will see
`typeof(v)`	Internal type.	`[1] "double"`
`mode(v)`	Broad type.	`[1] "numeric"`
`storage.mode(v)`	Storage type.	`[1] "double"`
`str(v)`	Structure summary: num means numeric.	`num 3.5`
`str("hi")`	chr means character.	`chr "hi"`
`str(TRUE)`	logi means logical.	`logi TRUE`
`str(5L)`	int means integer.	`int 5`
`inherits(v, "numeric")`	Does v belong to the numeric class?	`[1] TRUE`
`class(NULL)`	Even nothing has a class.	`[1] "NULL"`
`class(mean)`	Functions have a type too.	`[1] "function"`
`class(matrix(1:4, 2))`	A peek ahead: a matrix (a grid of numbers) has two class names. You will meet matrices in the data structures notes.	`[1] "matrix" "array"`

3.12 Converting between types

Sometimes you have data in the wrong type: numbers that arrived as text, or text you want as numbers. The as.something() functions convert them. If a conversion is impossible, R gives NA with a warning instead of crashing.

Code (type this)	What it does, in simple words	What you will see
`as.numeric("42")`	Text that looks like a number becomes a real number.	`[1] 42`
`as.numeric("abc")`	Impossible, so R gives NA and warns "NAs introduced by coercion".	`[1] NA`
`as.integer(3.9)`	Chops decimals off.	`[1] 3`
`as.character(99)`	Number becomes text.	`[1] "99"`
`as.character(TRUE)`	Logical becomes text.	`[1] "TRUE"`
`as.logical("TRUE")`	Text "TRUE" becomes a real TRUE.	`[1] TRUE`
`as.logical("yes")`	R does not understand "yes", so NA.	`[1] NA`
`as.logical(0)`	Zero counts as FALSE.	`[1] FALSE`
`as.logical(5)`	Any other number counts as TRUE.	`[1] TRUE`
`as.numeric(TRUE)`	TRUE becomes 1.	`[1] 1`
`as.complex(2)`	A normal number becomes complex, with imaginary part 0.	`[1] 2+0i`
`as.Date("2026-01-15")`	Text becomes a real Date.	`[1] "2026-01-15"`
`as.numeric(as.Date("1970-01-02"))`	Dates count days since 1 January 1970, so the 2nd is day 1.	`[1] 1`
`"5" + 3`	You cannot do maths with text, even if it looks like a number. (Left out of the ready-	`Error in "5" + 3 : non-numeric`

Code (type this)	What it does, in simple words	What you will see
	made R file.)	`argument to binary operator`
`as.numeric("5") + 3`	Convert first, then calculate.	`[1] 8`

When R converts for you (coercion)

A list in R can only hold one type. If you mix types, R quietly converts them all to the most flexible type. The order of flexibility from least to most is: logical, then integer, then numeric, then character.

Code (type this)	What it does, in simple words	What you will see
`c(1, "a", TRUE)`	Mixing a number, text and a logical. Everything becomes text, because text is the most flexible type.	`[1] "1" "a" "TRUE"`
`c(1, TRUE, FALSE)`	A number and logicals mix to numbers: TRUE becomes 1, FALSE becomes 0.	`[1] 1 1 0`
`c(1L, 2.5)`	An integer and a decimal mix to numeric.	`[1] 1.0 2.5`
`class(c(1L, 2.5))`	Confirm it is numeric.	`[1] "numeric"`
`class(c(TRUE, 5L))`	A logical and an integer mix to integer.	`[1] "integer"`

> **In plain words:** Think of it as a bus where everyone must wear the same uniform. If one passenger is text, everyone has to wear the text uniform.

3.13 Quick reference: data type cheat sheet

Type	Create it	Test / convert	Handy extras
Numeric	x <- 3.14	is.numeric(), as.numeric()	round(), sqrt(), abs(), %% , %/%
Integer	x <- 5L	is.integer(), as.integer()	.Machine$integer.max
Character	x <- "hi"	is.character(), as.character()	nchar(), paste(), toupper(), substr(), gsub()
Logical	x <- TRUE	is.logical(), as.logical()	any(), all(), which(), sum()
Complex	x <- 3+4i	is.complex(), as.complex()	Re(), Im(), Mod(), Conj()
Date	as.Date("2026-11-05")	class(), as.Date()	format(), weekdays(), seq()
Factor	factor(c("a","b"))	is.factor(), as.factor()	levels(), table(), nlevels()
Missing	NA	is.na()	na.rm = TRUE, ifelse()

3.14 Common beginner mistakes with data types

- **Quotes turn numbers into text.** "25" is text, 25 is a number. Maths only works on the number.
- **Comparing decimals with ==.** Use all.equal() or round() instead.
- **Testing for missing values with == NA.** Use is.na().
- **Forgetting na.rm = TRUE.** One missing value makes sum() and mean() answer NA.

- **Using as.numeric() on a factor.** Convert to text first with as.character().
- **Expecting integer results from /.** Division always gives a decimal-capable result; use %/% for whole-number division.
- **Mixing types in a list without noticing.** One piece of text turns the whole list into text.

3.15 Practice time

Try these on your own first. Answers are at the end of this document.

- Exercise 1: Create a variable price holding 249.99. Check its class and typeof. Round it to the nearest whole number.
- Exercise 2: Store the text "data science" in a variable. Print it in capital letters, count its characters, and replace "science" with "analysis".
- Exercise 3: Create the vector c(15, 22, 8, 31, 4). Without using a loop, count how many values are above 10 and list them.
- Exercise 4: How many days are there between 1 January 2026 and 25 December 2026?
- Exercise 5: Make the vector c(4, NA, 10, 6). Find its mean, ignoring the missing value, then replace the missing value with 0.
- Exercise 6: Predict the class of c(1, "2", TRUE) before running it. Then check.

Answers to the practice exercises

Code (type this)	What it does, in simple words	What you will see

`# Exercise 1`
`price <- 249.99`
`class(price)`
`typeof(price)`
`round(price)`
	Make the variable, check its type (numeric and double), then round it.	`[1] "numeric"`
		`[1] "double"`
		`[1] 250`

`# Exercise 2`
`topic <- "data science"`
`toupper(topic)`
`nchar(topic)`
`sub("science", "analysis", topic)`
	Capitals, character count (12 including the space), and a replacement.	`[1] "DATA SCIENCE"`
		`[1] 12`
		`[1] "data analysis"`

`# Exercise 3`
`v <- c(15, 22, 8, 31, 4)`
`sum(v > 10)`
`v[v > 10]`
	Compare to 10 (TRUE/FALSE for each), count the TRUEs with sum(), then filter with the same test.	`[1] 3`
		`[1] 15 22 31`

`# Exercise 4`
`as.Date("2026-12-25") -`
`as.Date("2026-01-01")`
	Subtract the two dates.	`Time difference of 358 days`

`# Exercise 5`
`w <- c(4, NA, 10, 6)`
`mean(w, na.rm = TRUE)`
`ifelse(is.na(w), 0, w)`
	The average of 4, 10 and 6 is 6.666667. The second line swaps the NA for 0.	`[1] 6.666667`
		`[1] 4 0 10 6`

`# Exercise 6` `class(c(1, "2", TRUE))`	Number, text and logical all mix to text.	`[1] "character"`

Where to go next

You now know how to talk to R, store things in variables, and recognise every basic type of data. Next up are operators, then data structures such as vectors, matrices, lists and data frames, and then control statements. Every one of them builds on what you just learnt.

Keep practising a little each day. Fifteen minutes of typing and experimenting every day will teach you more than a long session once a week. You are doing great.